

Retail Business Performance Analysis

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files
 Sample-Superstore.csv

Sample-Superstore.csv(text/csv) - 2477201 bytes, last modified: 9/28/2026 - 100% done
 Saving Sample-Superstore.csv to Sample-Superstore (1).csv

```
import pandas as pd
```

```
df = pd.read_csv("Sample-Superstore.csv")
```

```
print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Dataset loaded successfully!
 Rows: 9994
 Columns: 22

```
print(df.columns.tolist())
```

```
['csvbase_row_id', '\uffeffRow ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity']
```

```
df.head()
```

	csvbase_row_id	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	
0		1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600	1
1		2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	1
2		3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200	1
3		4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	1
4		5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680	1

5 rows × 22 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 22 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   csvbase_row_id      9994 non-null   int64
1   Row ID              9994 non-null   int64
2   Order ID            9994 non-null   object
3   Order Date          9994 non-null   object
4   Ship Date           9994 non-null   object
5   Ship Mode           9994 non-null   object
6   Customer ID         9994 non-null   object
7   Customer Name       9994 non-null   object
8   Segment             9994 non-null   object
9   Country             9994 non-null   object
10  City                9994 non-null   object
11  State               9994 non-null   object
12  Postal Code         9994 non-null   int64
13  Region              9994 non-null   object
14  Product ID          9994 non-null   object
15  Category            9994 non-null   object
16  Sub-Category        9994 non-null   object
17  Product Name        9994 non-null   object
18  Sales               9994 non-null   float64
19  Quantity            9994 non-null   int64
20  Discount            9994 non-null   float64
21  Profit              9994 non-null   float64
dtypes: float64(3), int64(4), object(15)
memory usage: 1.7+ MB
```

```
df.isnull().sum()
```

Ø	
csvbase_row_id	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
dtype: int64	

```
print("Duplicate rows:", df.duplicated().sum())
```

Duplicate rows: 0

```
# Remove the unnecessary first column
df = df.drop(columns=['csvbase_row_id'])

# Remove the hidden BOM character from column names
df.columns = df.columns.str.replace('\u00ff', '', regex=False)

# Display cleaned column names
print(df.columns.tolist())
```

['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Product

df.dtypes	
	Ø
Row ID	int64
Order ID	object
Order Date	object
Ship Date	object
Ship Mode	object
Customer ID	object
Customer Name	object
Segment	object
Country	object
City	object
State	object
Postal Code	int64
Region	object
Product ID	object
Category	object
Sub-Category	object
Product Name	object
Sales	float64
Quantity	int64
Discount	float64
Profit	float64
dtype: object	

```
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

print(df.dtypes)
```

```
Row ID          int64
Order ID        object
Order Date      datetime64[ns]
Ship Date       datetime64[ns]
Ship Mode       object
Customer ID     object
Customer Name   object
Segment        object
Country        object
City           object
State          object
Postal Code     int64
Region         object
Product ID     object
Category       object
Sub-Category   object
Product Name    object
Sales          float64
Quantity       int64
Discount       float64
Profit         float64
dtype: object
```

```
print("Order Date:", df['Order Date'].min(), "to", df['Order Date'].max())
print("Ship Date:", df['Ship Date'].min(), "to", df['Ship Date'].max())
```

```
Order Date: 2014-01-03 00:00:00 to 2017-12-30 00:00:00
Ship Date: 2014-01-07 00:00:00 to 2018-01-05 00:00:00
```

Double-click (or enter) to edit

```
import sqlite3

# Create SQLite database
conn = sqlite3.connect("retail_analysis.db")

print("Database created successfully!")
```

```
Database created successfully!
```

```
df.to_sql(
    "retail_sales",
    conn,
    if_exists="replace",
    index=False
)

print("Data successfully loaded into SQL table!")
```

```
Data successfully loaded into SQL table!
```

```
query = """
SELECT *
FROM retail_sales
LIMIT 10;
"""

result = pd.read_sql_query(query, conn)

result
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	
0	1	CA-2016-152156	2016-11-08 00:00:00	2016-11-11 00:00:00	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	2	CA-2016-152156	2016-11-08 00:00:00	2016-11-11 00:00:00	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	3	CA-2016-138688	2016-06-12 00:00:00	2016-06-16 00:00:00	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	4	US-2015-108966	2015-10-11 00:00:00	2015-10-18 00:00:00	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	5	US-2015-108966	2015-10-11 00:00:00	2015-10-18 00:00:00	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680
5	6	CA-2014-115812	2014-06-09 00:00:00	2014-06-14 00:00:00	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	FUR-FU-10001487	Furniture	Furnishings	Eldon Expressions Wood and Plastic Desk Access...	48.8600
6	7	CA-2014-115812	2014-06-09 00:00:00	2014-06-14 00:00:00	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	OFF-AR-10002833	Office Supplies	Art	Newell 322	7.2800
7	8	CA-2014-115812	2014-06-09 00:00:00	2014-06-14 00:00:00	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	TEC-PH-10002275	Technology	Phones	Mitel 5320 IP Phone VoIP phone	907.1520
8	9	CA-2014-115812	2014-06-09 00:00:00	2014-06-14 00:00:00	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	OFF-BI-10003910	Office Supplies	Binders	DXL Angle-View Binders with Locking Rings by S...	18.5040
9	10	CA-2014-115812	2014-06-09 00:00:00	2014-06-14 00:00:00	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	OFF-AP-10002892	Office Supplies	Appliances	Belkin F5C206VTEL 6 Outlet Surge	114.9000

```
query = """
SELECT COUNT(*) AS total_records
FROM retail_sales;
""""

pd.read_sql_query(query, conn)
```

total_records	
0	9994

```
query = """
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN [Order ID] IS NULL OR [Order ID] = '' THEN 1 ELSE 0 END) AS missing_order_id,
    SUM(CASE WHEN [Customer ID] IS NULL OR [Customer ID] = '' THEN 1 ELSE 0 END) AS missing_customer_id,
    SUM(CASE WHEN [Category] IS NULL OR [Category] = '' THEN 1 ELSE 0 END) AS missing_category,
    SUM(CASE WHEN [Sales] IS NULL THEN 1 ELSE 0 END) AS missing_sales,
    SUM(CASE WHEN [Profit] IS NULL THEN 1 ELSE 0 END) AS missing_profit
FROM retail_sales;
"""

pd.read_sql_query(query, conn)
```

	total_rows	missing_order_id	missing_customer_id	missing_category	missing_sales	missing_profit
0	9994	0	0	0	0	0

```
query = """
SELECT
    [Order ID],
    COUNT(*) AS order_count
FROM retail_sales
GROUP BY [Order ID]
HAVING COUNT(*) > 1
LIMIT 10;
"""

pd.read_sql_query(query, conn)
```

Order ID order_count			
0	CA-2014-100090	2	
1	CA-2014-100363	2	
2	CA-2014-100678	4	
3	CA-2014-100706	2	
4	CA-2014-100762	4	
5	CA-2014-100895	3	
6	CA-2014-100916	3	
7	CA-2014-101560	4	
8	CA-2014-101602	2	
9	CA-2014-101931	5	

<pre>query = """ SELECT [Category], COUNT(*) AS records FROM retail_sales GROUP BY [Category] ORDER BY records DESC; """ pd.read_sql_query(query, conn)</pre>			
Category records			
0	Office Supplies	6026	
1	Furniture	2121	
2	Technology	1847	

```
query = """
SELECT
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    SUM([Quantity]) AS total_quantity,
    COUNT(DISTINCT [Order ID]) AS total_orders,
    COUNT(DISTINCT [Customer ID]) AS total_customers
FROM retail_sales;
"""

pd.read_sql_query(query, conn)
```

	total_sales	total_profit	total_quantity	total_orders	total_customers	
0	2297200.86	286397.02	37873	5009	793	

```
query = """
SELECT
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales;
"""

pd.read_sql_query(query, conn)
```

	total_sales	total_profit	profit_margin_percentage	
0	2297200.86	286397.02	12.47	

```
query = """
SELECT
    [Category],
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    SUM([Quantity]) AS total_quantity,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales
GROUP BY [Category]
ORDER BY total_profit DESC;
"""

category_analysis = pd.read_sql_query(query, conn)

category_analysis
```

	Category	total_sales	total_profit	total_quantity	profit_margin_percentage	
0	Technology	836154.03	145454.95	6939	17.40	
1	Office Supplies	719047.03	122490.80	22906	17.04	
2	Furniture	741999.80	18451.27	8028	2.49	

```
query = """
SELECT
    [Category],
    [Sub-Category],
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    SUM([Quantity]) AS total_quantity,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales
GROUP BY [Category], [Sub-Category]
ORDER BY total_profit DESC;
"""

subcategory_analysis = pd.read_sql_query(query, conn)

subcategory_analysis
```

	Category	Sub-Category	total_sales	total_profit	total_quantity	profit_margin_percentage	
0	Technology	Copiers	149528.03	55617.82	234	37.20	 
1	Technology	Phones	330007.05	44515.73	3289	13.49	
2	Technology	Accessories	167380.32	41936.64	2976	25.05	
3	Office Supplies	Paper	78479.21	34053.57	5178	43.39	
4	Office Supplies	Binders	203412.73	30221.76	5974	14.86	
5	Furniture	Chairs	328449.10	26590.17	2356	8.10	
6	Office Supplies	Storage	223843.61	21278.83	3158	9.51	
7	Office Supplies	Appliances	107532.16	18138.01	1729	16.87	
8	Furniture	Furnishings	91705.16	13059.14	3563	14.24	
9	Office Supplies	Envelopes	16476.40	6964.18	906	42.27	
10	Office Supplies	Art	27118.79	6527.79	3000	24.07	
11	Office Supplies	Labels	12486.31	5546.25	1400	44.42	
12	Technology	Machines	189238.63	3384.76	440	1.79	
13	Office Supplies	Fasteners	3024.28	949.52	914	31.40	
14	Office Supplies	Supplies	46673.54	-1189.10	647	-2.55	
15	Furniture	Bookcases	114880.00	-3472.56	868	-3.02	
16	Furniture	Tables	206965.53	-17725.48	1241	-8.56	

Next steps: [Generate code with subcategory_analysis](#) [New interactive sheet](#)

```
query = """
SELECT
    [Region],
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    SUM([Quantity]) AS total_quantity,
    COUNT(DISTINCT [Order ID]) AS total_orders,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales
GROUP BY [Region]
ORDER BY total_profit DESC;
"""

region_analysis = pd.read_sql_query(query, conn)

region_analysis
```

	Region	total_sales	total_profit	total_quantity	total_orders	profit_margin_percentage	
0	West	725457.82	108418.45	12266	1611	14.94	 
1	East	678781.24	91522.78	10618	1401	13.48	
2	South	391721.91	46749.43	6209	822	11.93	
3	Central	501239.89	39706.36	8780	1175	7.92	

Next steps: [Generate code with region_analysis](#) [New interactive sheet](#)

```
query = """
SELECT
    [Category],
    ROUND(AVG([Discount]) * 100, 2) AS average_discount_percentage,
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales
GROUP BY [Category]
ORDER BY average_discount_percentage DESC;
"""

discount_analysis = pd.read_sql_query(query, conn)

discount_analysis
```

Category average_discount_percentage total_sales total_profit profit_margin_percentage						
0	Furniture	17.39	741999.80	18451.27	2.49	
1	Office Supplies	15.73	719047.03	122490.80	17.04	
2	Technology	13.23	836154.03	145454.95	17.40	

Next steps: [Generate code with discount_analysis](#) [New interactive sheet](#)

```
query = """
SELECT
    ROUND([Discount] * 100, 0) AS discount_percentage,
    COUNT(*) AS transactions,
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    ROUND(
        (SUM([Profit]) * 100.0) / SUM([Sales]),
        2
    ) AS profit_margin_percentage
FROM retail_sales
GROUP BY [Discount]
ORDER BY [Discount];
"""

discount_level_analysis = pd.read_sql_query(query, conn)
discount_level_analysis
```

discount_percentage transactions total_sales total_profit profit_margin_percentage						
0	0.0	4798	1087908.47	320987.60	29.51	
1	10.0	94	54369.35	9029.18	16.61	
2	15.0	52	27558.52	1418.99	5.15	
3	20.0	3657	764594.37	90337.31	11.82	
4	30.0	227	103226.65	-10369.28	-10.05	
5	32.0	27	14493.46	-2391.14	-16.50	
6	40.0	206	116417.78	-23057.05	-19.81	
7	45.0	11	5484.97	-2493.11	-45.45	
8	50.0	66	58918.54	-20506.43	-34.80	
9	60.0	138	6644.70	-5944.66	-89.46	
10	70.0	418	40620.28	-40075.36	-98.66	
11	80.0	300	16963.76	-30539.04	-180.03	

Next steps: [Generate code with discount_level_analysis](#) [New interactive sheet](#)

```
query = """
SELECT
    [Category],
    [Sub-Category],
    [Product Name],
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    SUM([Quantity]) AS total_quantity,
    ROUND(AVG([Discount]) * 100, 2) AS average_discount_percentage
FROM retail_sales
GROUP BY
    [Category],
    [Sub-Category],
    [Product Name]
HAVING SUM([Profit]) < 0
ORDER BY total_profit ASC
LIMIT 20;
"""

loss_making_products = pd.read_sql_query(query, conn)
loss_making_products
```

	Category	Sub-Category	Product Name	total_sales	total_profit	total_quantity	average_discount_percentage	
0	Technology	Machines	Cubify CubeX 3D Printer Double Head Print	11099.96	-8879.97	9	53.33	
1	Technology	Machines	Lexmark MX611dhe Monochrome Laser Printer	16829.90	-4589.97	18	40.00	
2	Technology	Machines	Cubify CubeX 3D Printer Triple Head Print	7999.98	-3839.99	4	50.00	
3	Furniture	Tables	Chromcraft Bull-Nose Wood Oval Conference Tabl...	9917.64	-2876.12	27	28.00	
4	Furniture	Tables	Bush Advantage Collection Racetrack Conference...	9544.73	-1934.40	33	35.00	
5	Office Supplies	Binders	GBC DocuBind P400 Electric Binding System	17965.07	-1878.17	27	45.00	
6	Technology	Machines	Cisco TelePresence System EX90 Videoconferenci...	22638.48	-1811.08	6	50.00	
7	Office Supplies	Supplies	Martin Yale Chadless Opener Electric Letter Op...	16656.20	-1299.18	22	10.00	
8	Furniture	Tables	Balt Solid Wood Round Tables	6518.75	-1201.06	19	20.00	
9	Furniture	Tables	BoxOffice By Design Rectangular and Half-Moon ...	1706.25	-1148.44	15	48.33	
10	Furniture	Tables	Riverside Furniture Oval Coffee Table, Oval En...	4446.18	-1147.40	25	30.00	
11	Technology	Machines	Epson TM-T88V Direct Thermal Printer - Monochr...	1212.70	-1057.23	10	55.00	
12	Furniture	Chairs	Hon 2090 "Pillow Soft" Series Mid Back Swivel/...	5282.42	-989.05	24	21.67	
13	Furniture	Bookcases	O'Sullivan 4-Shelf Bookcase in Odessa Pine	2740.20	-975.10	37	42.14	
14	Furniture	Tables	Bretford "Just In Time" Height-Adjustable Mult...	5634.90	-964.19	17	22.50	
15	Technology	Machines	Zebra GK420t Direct Thermal/Thermal Transfer P...	703.71	-938.28	6	70.00	
16	Office Supplies	Appliances	3.6 Cubic Foot Counter Height Office Refrigerator	2946.20	-872.08	18	52.00	
17	Furniture	Tables	Bevis Oval Conference Table, Walnut	6942.07	-856.01	36	27.50	
18	Office Supplies	Storage	Tennsco Single-Tier Lockers	9158.30	-825.75	28	7.50	
19	Furniture	Tables	BPI Conference Tables	2241.87	-795.97	26	37.00	

Next steps: [Generate code with loss_making_products](#) [New interactive sheet](#)

```
query = """
SELECT
    [Category],
    [Sub-Category],
    [Product Name],
    SUM([Quantity]) AS total_quantity,
    ROUND(SUM([Sales]), 2) AS total_sales,
    ROUND(SUM([Profit]), 2) AS total_profit,
    ROUND(AVG([Discount]) * 100, 2) AS average_discount_percentage
FROM retail_sales
GROUP BY
    [Category],
    [Sub-Category],
    [Product Name]
ORDER BY total_quantity ASC
LIMIT 20;
"""

slow_moving_products = pd.read_sql_query(query, conn)
slow_moving_products
```

	Category	Sub-Category	Product Name	total_quantity	total_sales	total_profit	average_discount_percentage	
0	Furniture	Bookcases	Bush Saratoga Collection 5-Shelf Bookcase, Han...	1	119.83	-12.69	15.0	
1	Furniture	Chairs	Global Enterprise Series Seating Low-Back Swiv...	1	207.18	25.90	20.0	
2	Office Supplies	Art	Boston 1900 Electric Pencil Sharpener	1	14.98	4.49	0.0	
3	Office Supplies	Paper	Xerox 20	1	6.48	3.11	0.0	
4	Technology	Machines	Penpower WorldCard Pro Card Scanner	1	91.47	-1.83	50.0	
5	Furniture	Furnishings	Ultra Commercial Grade Dual Valve Door Closer	2	63.82	9.57	20.0	
6	Furniture	Tables	Barricks Non-Folding Utility Table with Steel ...	2	136.46	15.35	20.0	
7	Office Supplies	Appliances	Eureka Disposable Bags for Sanitaire Vibra Gro...	2	1.62	-4.47	80.0	
8	Office Supplies	Art	Avery Hi-Liter Pen Style Six-Color Fluorescent...	2	7.70	3.16	0.0	
9	Office Supplies	Envelopes	Grip Seal Envelopes	2	7.07	2.39	20.0	
10	Office Supplies	Envelopes	Jiffy Padded Mailers with Self-Seal Closure	2	29.81	10.81	20.0	
11	Office Supplies	Envelopes	Multimedia Mailers	2	325.86	149.90	0.0	
12	Office Supplies	Labels	Avery 5	2	5.76	2.82	0.0	
13	Office Supplies	Paper	Xerox 1969	2	11.56	5.66	0.0	
14	Office Supplies	Paper	Xerox 1984	2	12.96	6.35	0.0	
15	Office Supplies	Paper	Xerox 1989	2	7.97	2.69	20.0	
16	Technology	Accessories	Memorex Mini Travel Drive 4 GB USB 2.0 Flash D...	2	13.62	3.57	20.0	
17	Technology	Machines	Canon imageCLASS MF7460 Monochrome Digital Las...	2	3991.98	1995.99	0.0	
18	Technology	Machines	Hewlett-Packard Deskjet F4180 All-in-One Color...	2	101.99	-71.40	70.0	
19	Technology	Machines	NeatDesk Desktop Scanner & Digital Filing System	2	479.99	56.00	40.0	

Next steps: [Generate code with slow_moving_products](#) [New interactive sheet](#)

```
python_df = df.copy()

python_df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600	2	
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200	2	
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680	2	
5 rows × 21 columns																				

python_df[['Sales', 'Quantity', 'Discount', 'Profit']].describe()

	Sales	Quantity	Discount	Profit	
count	9994.000000	9994.000000	9994.000000	9994.000000	
mean	229.858001	3.789574	0.156203	28.656896	
std	623.245101	2.225110	0.206452	234.260108	
min	0.444000	1.000000	0.000000	-6599.978000	
25%	17.280000	2.000000	0.000000	1.728750	
50%	54.490000	3.000000	0.200000	8.666500	
75%	209.940000	5.000000	0.200000	29.364000	
max	22638.480000	14.000000	0.800000	8399.976000	

```
correlation = python_df[['Sales', 'Quantity', 'Discount', 'Profit']].corr()

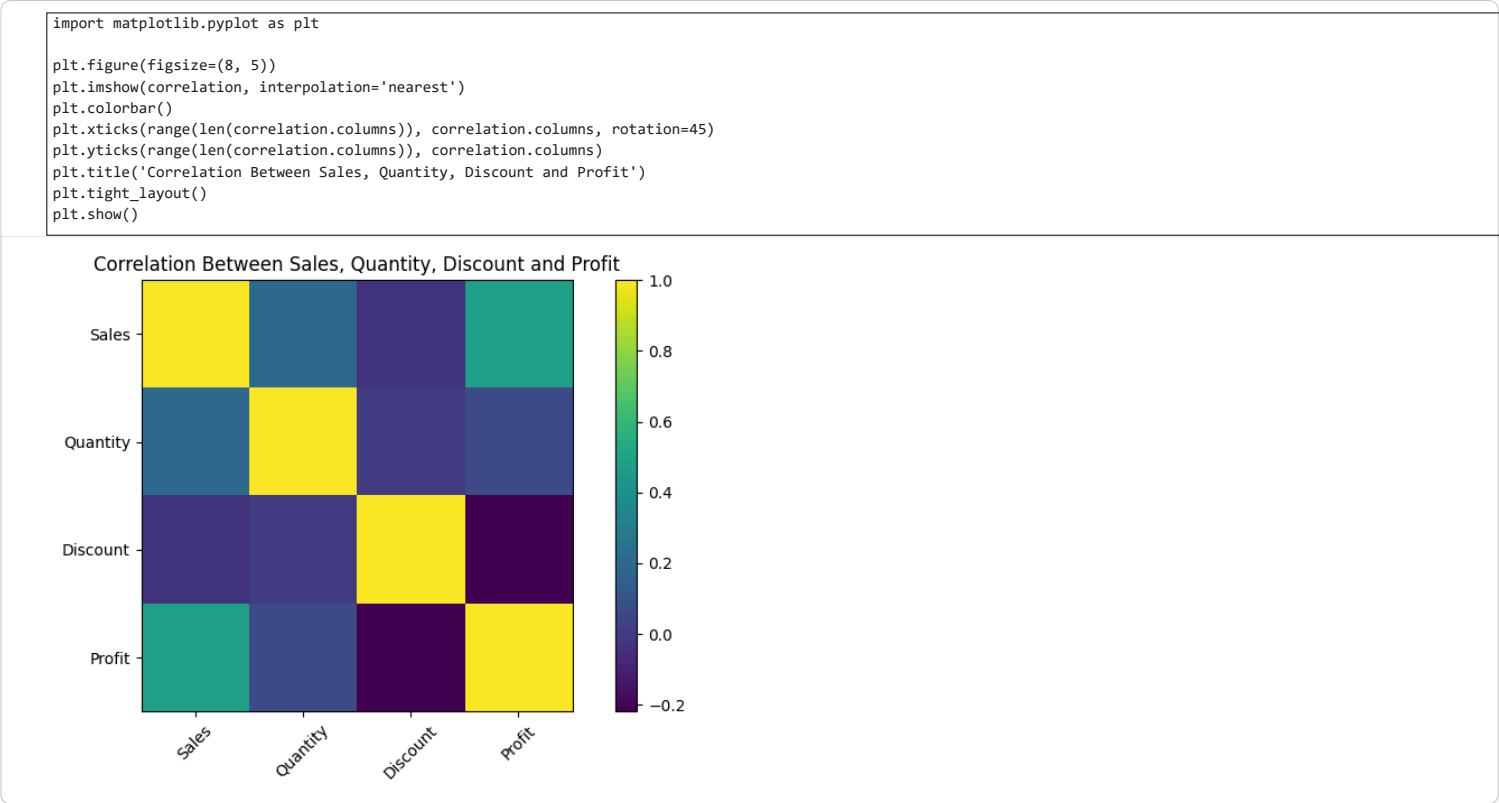
correlation
```

	Sales	Quantity	Discount	Profit	
Sales	1.000000	0.200795	-0.028190	0.479064	
Quantity	0.200795	1.000000	0.008623	0.066253	
Discount	-0.028190	0.008623	1.000000	-0.219487	
Profit	0.479064	0.066253	-0.219487	1.000000	

Next steps:

Generate code with correlation

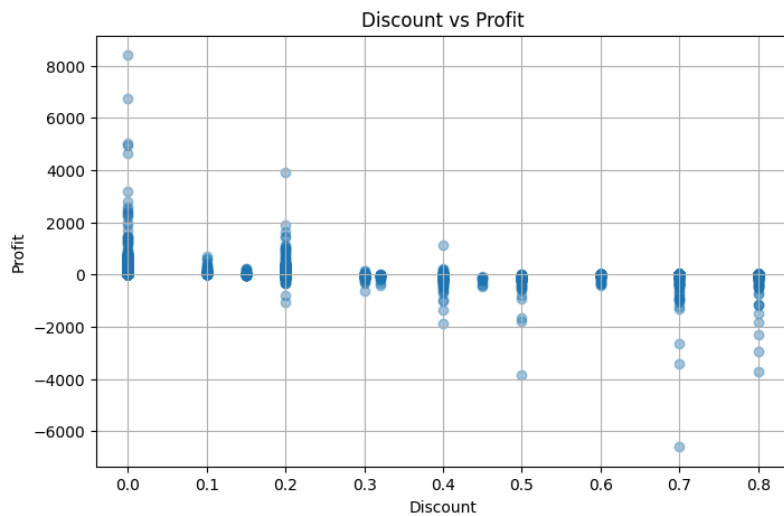
New interactive sheet



```
plt.figure(figsize=(8, 5))

plt.scatter(
    python_df['Discount'],
    python_df['Profit'],
    alpha=0.4
)

plt.xlabel('Discount')
plt.ylabel('Profit')
plt.title('Discount vs Profit')
plt.grid(True)
plt.show()
```



```
category_profit = (
    python_df.groupby('Category')['Profit']
    .sum()
    .sort_values(ascending=False)
)

category_profit
```

Profit	
Category	
Technology	145454.9481
Office Supplies	122490.8008
Furniture	18451.2728

dtype: float64

```
plt.figure(figsize=(8, 5))

category_profit.plot(kind='bar')

plt.xlabel('Category')
plt.ylabel('Total Profit')
plt.title('Total Profit by Category')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

